

Studiu Comparativ al Algoritmilor de Sortare

Modulul: Metode și Practici în Informatică (MPI)

David Tudor

Universitatea de Vest din Timișoara
Facultatea de Informatică

`david.tudor06@e-uvv.ro`

Mai 2026

1. Introducere și Motivația Proiectului

Context general

Gestionarea eficientă a volumelor de date în baze de date, motoare de căutare și sistemele de operare moderne.

De ce acest proiect?

- Analiza decalajului dintre complexitatea teoretică ($O(n^2)$, $O(n \log n)$) și performanța reală (empirică).
- Influența arhitecturilor hardware moderne asupra execuției codului.
- Evaluarea comportamentului algoritmilor în funcție având în vedere starea inițială a datelor.

2. Prezentarea și Descrierea Algoritmilor Studiați

Studiul acoperă două paradigme majore cu impact direct asupra resurselor hardware:

Algoritmi Quadratici ($O(n^2)$)

- **Bubble Sort:** Compară și inversează elemente adiacente repetat până la ordonare.
- **Selection Sort:** Selectează iterativ elementul minim și îl plasează la indexul curent.
- **Insertion Sort:** Inserează elementul curent pe poziția corectă într-o sub-listă deja sortată.

Algoritmi Log-Liniari ($O(n \log n)$)

- **Merge Sort:** Divide lista în jumătăți, le sortează recursiv și le interclasează (stabil, consumă memorie extra).
- **Quick Sort:** Folosește un pivot pentru partiționarea listei în sub-probleme independente (eficient „in-place”).

3. Arhitectura Aplicației de Test (Benchmark)

Sistemul a fost dezvoltat modular în **C++**, fiind structurat în trei componente cheie:

Algorithm.h

Implementarea curată a celor 5 algoritmi analizați:
Bubble, Selection, Insertion, Merge și Quick Sort.

Benchmark.h

Măsurarea timpului real de execuție în milisecunde
utilizând biblioteca standard <chrono>.

Generators.h

Generează seturi de date cu 4 distribuții native:
Random, Sorted, Reverse și Flat (Egal).

4. Rezultate Empirice (10.000 de Elemente)

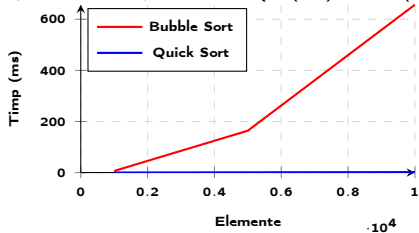
Distribuție	Bubble	Selection	Insertion	Merge	Quick
Random	657,44 ms	188,59 ms	213,15 ms	3,57 ms	1,89 ms
Sorted	0,03 ms	188,06 ms	0,08 ms	1,90 ms	0,56 ms
Reverse	749,53 ms	273,96 ms	544,90 ms	1,90 ms	0,33 ms
Flat (Egal)	514,31 ms	253,69 ms	181,93 ms	1,90 ms	<i>47,39 ms</i>

Tabela: Timpii de execuție obținuți în urma testelor practice

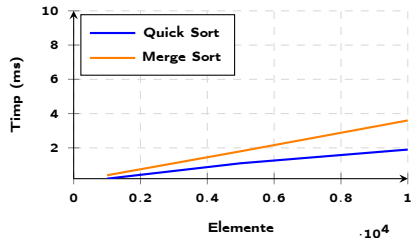
- **Prăpastie uriașă:** Pe date aleatoare, algoritmi eficienți ($O(n \log n)$) devin de sute de ori mai rapizi decât cei clasici quadratici.

5. Reprezentarea Grafică a Scalabilității

Diferența de Complexitate ($O(n^2)$ vs $O(n \log n)$)



Detaliu: Comportamentul Algoritmilor Eficienți



6. Analiză Grafică și Scalabilitate: Fereastra Micro vs. Large

Analiza evoluției timpilor de rulare evidențiază comportamente complet diferite în funcție de volumul de date:

Fereastra Micro (10 - 250 elemente)

- **Insertion Sort** este lider absolut datorită overhead-ului structural minim.
- Toți timpii se mențin sub **0.5 ms**.
- Simplitatea codului bate complexitatea teoretică asimptotică la volume mici.

Fereastra Large (5.000 - 20.000 elemente)

- Timpul pentru **Bubble Sort** explodează la peste **2600 ms**.
- **Quick** și **Merge Sort** rămân stabile, sub pragul de **8 ms**.
- Se confirmă practic superioritatea algoritmilor de tip Divide et Impera.

7. Investigarea Anomaliilor de Performanță

În faza de testare am identificat două comportamente non-standard extrem de importante:

❶ Degradarea Quick Sort pe liste constante (Flat):

- Timpul a crescut neașteptat la **47,39 ms** din cauza alegerii nefavorabile a pivotului.
- Determină partiționarea dezechilibrată și forțează complexitatea spre cazul defavorabil $O(n^2)$.

❷ Eficiența extremă a Insertion Sort pe liste sortate:

- A finalizat în doar **0,08 ms**, devansând chiar și Quick Sort (*0,56 ms*) și Merge Sort (*1,90 ms*).
- Explicație: Numărul minim de deplasări în memorie ($O(n)$ în scenariul ideal).

8. Concluzii și Recomandări Practice

- **Nu există un algoritm ideal universal:** Optimizarea depinde critic de structura datelor de intrare.
- **Algoritmii hibridi:** Sisteme moderne (ex: `std::sort` din C++) combină Quick Sort, Heap Sort și Insertion Sort (Introsort) pentru a evita anomaliile observate.
- **Punct critic:** Pentru seturi care depășesc 1.000 de elemente, utilizarea metodelor clasice ($O(n^2)$) în aplicații în timp real devine inacceptabilă.

Vă mulțumesc pentru atenție!

Întrebări și discuții...